UNIVERSIDAD NACIONAL DEL SUR
Departamento de Ciencias e Ingeniería de la Computación
Compiladores e Intérpretes — Segundo Cuatrimestre de 2026

Compilador de MiniJava

Etapa 2 — Analizador Sintáctico

Descendente Recursivo Predictivo

Agustín Paul
L.U. 147524

Informe de entrega

Transformación de la gramática a LL(1), paso a paso.

19 de septiembre de 2026

Índice

1. Cómo compilar y ejecutar **1**

2. Logros **1**

3. Transformación de la gramática **1**
3.1. Método 1
3.2. Diagnóstico 2
3.3. Eliminación de recursión a izquierda 3
3.4. Factorización 4
3.5. El núcleo 8
3.6. Logros 8
3.7. Ambigüedades que quedan 14

4. Gramática final **14**

5. El analizador sintáctico **19**
5.1. Diseño 19
5.2. Errores que detecta 19

6. Decisiones **20**
6.1. Sobre puntos consultados 20
6.2. Otras 20

1 Cómo compilar y ejecutar

Proyecto Maven con Java 25. Desde la carpeta `compilador/`:

```
$ mvn package          # compila, corre la suite y arma target/Compilador.jar
$ java -jar target/Compilador.jar programa1.java
Compilacion Exitosa

[SinErrores]
```

El fuente es el único argumento y puede tener cualquier extensión. `--etapa=1` corre sólo el analizador léxico de la Etapa 1. El compilador corre siempre con todos sus logros.

2 Logros

Logro	**Dónde está**
Variables locales Clásicas E2	L4 (§3.6).
Visibilidad Mejorada E2	L6 (§3.6). Agrega el token **private**.
Fors! E2	L7 (§3.6). Agrega el token **for**.
Operador Ternario E2	L3 (§3.6). Agrega el token **?**.
Operadores Posifijos E2	L2 (§3.6).
Atributos Inicializados	L5 (§3.6).
Inicializacion de Arreglos E2	L8 (§3.6).
Genericidad Avanzada E2	L9 (§3.6).
Recuperación de Errores Modo Pánico	Implementación (§5.2).
Floats I (Etapa 1)	L1 (§3.6).

También se persiguen *Entrega Anticipada*, *Imbatibilidad* y *Well Done!*, que no dependen del código.

3 Transformación de la gramática

3.1 Método

Criterio LL(1). Para todo $A \to \beta_1 \mid \cdots \mid \beta_n$, $\text{Primeros}(\beta_i) \cap \text{Primeros}(\beta_j) = \varnothing$ si $i \neq j$.

Orden. Primero se elimina la recursión a izquierda y después se factoriza, porque eliminar la recursión puede dejar producciones sin factorizar (R3).

Técnicas.

RI	Eliminación de recursión a izquierda directa: $A \to A\alpha_1 \mid \cdots \mid A\alpha_n \mid \beta_1 \mid \cdots \mid \beta_m$ pasa a $A \to \beta_1 R \mid \cdots \mid \beta_m R$ y $R \to \alpha_1 R \mid \cdots \mid \alpha_n R \mid \epsilon$.
FD	Factorización a izquierda directa: $X \to \alpha\beta_1 \mid \cdots \mid \alpha\beta_n \mid \gamma_1 \mid \cdots \mid \gamma_m$ pasa a $X \to \alpha R \mid \gamma_1 \mid \cdots \mid \gamma_m$ y $R \to \beta_1 \mid \cdots \mid \beta_n$.
SU	Subir: reemplazar un no terminal por sus partes derechas para exponer el prefijo común.
FP	Factorización profunda: **SU** hasta que el prefijo sea directo, y después **FD**.
AG	Agrupar: reemplazar alternativas, o una cola repetida, por un no terminal nuevo que deriva exactamente eso. Es la inversa de **SU**: no cambia el lenguaje ni hace falta para LL(1). Se usa para que cada punto de decisión conserve un nombre, porque las reglas tienen un significado que el semántico necesita.

Conjuntos que se repiten. $\mathcal{T}$ = Primeros(`<Tipo>`) = {**boolean**, **char**, **int**, **idClase**, **idGen**}; $\mathcal{E}$ = Primeros(`<Expresion>`) = {**!**, **(**, **+**, **-**, **charLiteral**, **false**, **idClase**, **idMetVar**, **intLiteral**, **new**, **null**, **stringLiteral**, **this**, **true**}; $\mathcal{P}$ = Primeros(`<Primario>`) = {**(**, **idClase**, **idMetVar**, **new**, **stringLiteral**, **this**}. Desde L1, $\mathcal{T}$ incluye **float** y $\mathcal{E}$, **floatLiteral**.

3.2 Diagnóstico

La gramática de la cátedra tiene **16 conflictos en 10 no terminales**.

#	No terminal	Defecto	Intersección	Paso
1	`<Miembro>`	prefijo común no directo	$\mathcal{T}$	F6
2	`<ListaArgsFormales>`	recursión a izquierda	$\mathcal{T}$	R1
3	`<Sentencia>`	`<Asignacion>` y `<Llamada>` idénticos	$\mathcal{E}$	F2
4	`<If>`	prefijo común	{**if**}	F3
5	`<Expresion>`	prefijo común	$\mathcal{E}$	F1
6	`<ExpresionCompuesta>`	recursión a izquierda	$\mathcal{E}$	R2
7–12	`<Referencia>`	recursión a izquierda, seis pares	$\mathcal{P}$	R3
13	`<Primario>`	prefijo común no directo	{**idMetVar**}	F7
14	`<Primario>`	prefijo común a dos niveles	{**new**}	F8
15	`<DimensionesConTamanio>`	prefijo común	{**[**}	F4
16	`<ListaExps>`	prefijo común	$\mathcal{E}$	F5

Ambigüedades. Un conflicto LL(1) no implica ambigüedad, que es tener dos árboles para una misma cadena. La base tiene tres: (1) `<Asignacion>` y `<Llamada>` tienen el mismo cuerpo, así que toda sentencia-expresión tiene dos árboles; se elimina en F2. (2) El *dangling else*, `if (a) if (b) s1; else s2;`; sobrevive a F3. (3) En `new int[3][4]` el segundo **[** puede ser otra dimensión o un acceso al arreglo creado; no es un conflicto de Primeros de la base y aparece en F4. En los otros catorce conflictos las alternativas derivan lenguajes disjuntos.

3.3 Eliminación de recursión a izquierda

R1 — `<ListaArgsFormales>`

Original

`<ListaArgsFormales> ::= <ArgFormal> | <ListaArgsFormales> , <ArgFormal>`

Conflicto. Las dos alternativas tienen Primeros $\mathcal{T}$.

Paso. **RI** con β_1 = `<ArgFormal>`, α_1 = `, <ArgFormal>` y R = `<RestoListaArgsFormales>`.

Resultado

`<ListaArgsFormales> ::= <ArgFormal> <RestoListaArgsFormales>`

`<RestoListaArgsFormales> ::= , <ArgFormal> <RestoListaArgsFormales>` | ϵ

$\{$`,`$\}$ contra $\{\epsilon\}$. ✓

R2 — `<ExpresionCompuesta>`

Original

```
<ExpresionCompuesta> ::= <ExpresionCompuesta> <OperadorBinario>
    <ExpresionBasica> | <ExpresionBasica>
```

Conflicto. Las dos alternativas tienen Primeros $\mathcal{E}$.

Paso. **RI** con β_1 = `<ExpresionBasica>`, α_1 = `<OperadorBinario> <ExpresionBasica>` y R = `<RestoExpresionCompuesta>`.

Resultado

`<ExpresionCompuesta> ::= <ExpresionBasica> <RestoExpresionCompuesta>`

```
<RestoExpresionCompuesta> ::= <OperadorBinario> <ExpresionBasica>
    <RestoExpresionCompuesta> | ϵ
```

Los trece operadores binarios contra $\{\epsilon\}$. ✓ Siguen en un solo nivel, como en la base.

R3 — `<Referencia>`

Original

```
<Referencia> ::= <Primario> | <Referencia> <VarEncadenada> | <Referencia>
    <MetodoEncadenado> | <Referencia> <AccesoArreglo>
<VarEncadenada> ::= . idMetVar
<MetodoEncadenado> ::= . idMetVar <ArgsActuales>
```

Conflicto. Las cuatro alternativas tienen Primeros $\mathcal{P}$: seis pares.

Paso 1. **RI** con β_1 = `<Primario>`, α_1 = `<VarEncadenada>`, α_2 = `<MetodoEncadenado>`, α_3 = `<AccesoArreglo>` y R = `<RestoReferencia>`:

```
<Referencia> ::= <Primario> <RestoReferencia>
<RestoReferencia> ::= <VarEncadenada> <RestoReferencia> |
    <MetodoEncadenado> <RestoReferencia> | <AccesoArreglo>
    <RestoReferencia> | ϵ
```

Conflicto nuevo: las dos primeras alternativas tienen Primeros $\{$`.`$\}$.

Paso 2 (AG). Las dos comparten la cola <RestoReferencia>:

<RestoReferencia> ::= <Encadenado> <RestoReferencia> | <AccesoArreglo> <RestoReferencia> | ϵ

<Encadenado> ::= <VarEncadenada> | <MetodoEncadenado>

El conflicto pasa a <Encadenado>: {.} contra {.}.

Paso 3. **FP**: **SU** de <VarEncadenada> y <MetodoEncadenado>,

<Encadenado> ::= . idMetVar | . idMetVar <ArgsActuales>

y **FD** con α = . idMetVar, β_1 = ϵ, β_2 = <ArgsActuales> y R = <EncadenadoAtributoOMetodo>.

Resultado

<Referencia> ::= <Primario> <RestoReferencia>

<RestoReferencia> ::= <Encadenado> <RestoReferencia> | <AccesoArreglo> <RestoReferencia> | ϵ

<Encadenado> ::= . idMetVar <EncadenadoAtributoOMetodo>

<EncadenadoAtributoOMetodo> ::= <ArgsActuales> | ϵ

<RestoReferencia>: {.}, {[} y {ϵ}. <EncadenadoAtributoOMetodo>: {(} contra {ϵ}. ✓ Se pierden <VarEncadenada> y <MetodoEncadenado>; la distinción queda en <EncadenadoAtributoOMetodo>: vacío es el atributo y con argumentos, el método.

3.4 Factorización

F1 — <Expresion>

Original

<Expresion> ::= <ExpresionCompuesta> <OperadorAsignacion> <ExpresionCompuesta> | <ExpresionCompuesta>

Conflicto. Las dos alternativas tienen Primeros $\mathcal{E}$.

Paso. **FD** con α = <ExpresionCompuesta>, β_1 = <OperadorAsignacion> <ExpresionCompuesta>, $\beta_2 = \epsilon$ y R = <RestoExpresion>.

Resultado

<Expresion> ::= <ExpresionCompuesta> <RestoExpresion>

<RestoExpresion> ::= <OperadorAsignacion> <ExpresionCompuesta> | ϵ

{=} contra {ϵ}. ✓ El lado derecho del = sigue siendo una <ExpresionCompuesta>: `a = b = c` no pertenece al lenguaje, como en la base.

F2 — <Sentencia>

Original

<Sentencia> ::= ; | <Asignacion> ; | <Llamada> ; | <VarLocal> ; | <Return> ; | <If> | <While> | <Bloque>

<Asignacion> ::= <Expresion>

<Llamada> ::= <Expresion>

Conflicto. `<Asignacion>` **;** y `<Llamada>` **;** tienen Primeros $\mathcal{E}$, y además son la misma producción: la ambigüedad (1).

Paso 1. **SU** de `<Asignacion>` y `<Llamada>`: las dos alternativas quedan `<Expresion>` **;**, la misma producción dos veces. Como conjunto de producciones es una sola, y con ella hay un solo árbol.

Paso 2 (AG). `<SentenciaExpresion> ::= <Expresion>`.

Resultado

```
<Sentencia> ::= ; | <SentenciaExpresion> ; | <VarLocal> ; | <Return> ; | <If>
    | <While> | <Bloque>
<SentenciaExpresion> ::= <Expresion>
```

{**;**}, $\mathcal{E}$, {**var**}, {**return**}, {**if**}, {**while**} y {**{**}. ✓ Distinguir una asignación de una llamada exige el significado de la expresión: es del semántico.

F3 — `<If>`

Original

```
<If> ::= if ( <Expresion> ) <Sentencia> | if ( <Expresion> ) <Sentencia> else
    <Sentencia>
```

Conflicto. {**if**} contra {**if**}.

Paso. **FD** con $\alpha =$ **if (** `<Expresion>` **)** `<Sentencia>`, $\beta_1 = \epsilon$, $\beta_2 =$ **else** `<Sentencia>` y $R =$ `<ElseOpcional>`.

Resultado

```
<If> ::= if ( <Expresion> ) <Sentencia> <ElseOpcional>
<ElseOpcional> ::= else <Sentencia> | ϵ
```

{**else**} contra {ϵ}. ✓ Queda la ambigüedad (2): `<ElseOpcional>` deriva a ϵ y lo que le sigue puede empezar con **else** (§3.7).

F4 — `<DimensionesConTamanio>`

Original

```
<DimensionesConTamanio> ::= [ <Expresion> ] <DimensionesConTamanio> | [
    <Expresion> ]
```

Conflicto. {**[**} contra {**[**}.

Paso. **FD** con $\alpha =$ **[** `<Expresion>` **]**, $\beta_1 =$ `<DimensionesConTamanio>`, $\beta_2 = \epsilon$ y $R =$ `<DimensionesConTamanioOpcional>`.

Resultado

```
<DimensionesConTamanio> ::= [ <Expresion> ] <DimensionesConTamanioOpcional>
<DimensionesConTamanioOpcional> ::= <DimensionesConTamanio> | ϵ
```

{**[**} contra {ϵ}. ✓ Aparece la ambigüedad (3): lo que sigue a `<DimensionesConTamanioOpcional>`, por `<Primario>` y `<RestoReferencia>`, puede empezar con **[** (§3.7).

F5 — <ListaExps>

Original

<ListaExps> ::= <Expresion> | <Expresion> , <ListaExps>

Conflicto. Las dos alternativas tienen Primeros $\mathcal{E}$.

Paso. **FD** con α = <Expresion>, $\beta_1 = \epsilon$, β_2 = , <ListaExps> y R = <RestoListaExps>.

Resultado

<ListaExps> ::= <Expresion> <RestoListaExps>

<RestoListaExps> ::= , <ListaExps> | ϵ

{,} contra {ϵ}. ✓

F6 — <Miembro>

Original

<Miembro> ::= <Atributo> | <Metodo> | <Constructor>

<Atributo> ::= <Tipo> idMetVar ;

<Metodo> ::= <ModificadorOpcional> <TipoMetodo> idMetVar <ArgsFormales> <Bloque>

<Constructor> ::= public idClase <ArgsFormales> <Bloque>

<ModificadorOpcional> ::= static | ϵ

<TipoMetodo> ::= <Tipo> | void

Conflicto. Primeros(<Atributo>) = $\mathcal{T}$, Primeros(<Metodo>) = {static, void} $\cup \mathcal{T}$ y Primeros(<Constructor>) = {public}: <Atributo> y <Metodo> chocan en $\mathcal{T}$. El prefijo no es directo.

Paso 1. **SU** de <Atributo> y <Metodo>:

<Miembro> ::= <Tipo> idMetVar ; | <ModificadorOpcional> <TipoMetodo> idMetVar <ArgsFormales> <Bloque> | <Constructor>

Paso 2. **SU** de <ModificadorOpcional>:

<Miembro> ::= <Tipo> idMetVar ;

<Miembro> ::= static <TipoMetodo> idMetVar <ArgsFormales> <Bloque>

<Miembro> ::= <TipoMetodo> idMetVar <ArgsFormales> <Bloque>

<Miembro> ::= <Constructor>

Paso 3. **SU** de <TipoMetodo> en la tercera:

<Miembro> ::= <Tipo> idMetVar ;

<Miembro> ::= static <TipoMetodo> idMetVar <ArgsFormales> <Bloque>

<Miembro> ::= <Tipo> idMetVar <ArgsFormales> <Bloque>

<Miembro> ::= void idMetVar <ArgsFormales> <Bloque>

<Miembro> ::= <Constructor>

Paso 4. **FD** con α = <Tipo> idMetVar, β_1 = ;, β_2 = <ArgsFormales> <Bloque> y R = <AtributoOMetodo>:

<Miembro> ::= <Tipo> idMetVar <AtributoOMetodo>

<Miembro> ::= static <TipoMetodo> idMetVar <ArgsFormales> <Bloque>

<Miembro> ::= void idMetVar <ArgsFormales> <Bloque>

<Miembro> ::= <Constructor>

<AtributoOMetodo> ::= ; | <ArgsFormales> <Bloque>

<Miembro>: $\mathcal{T}$, {static}, {void} y {public}. <AtributoOMetodo>: {;} contra {(}. ✓

Paso 5 (AG). <CuerpoDeMetodo> ::= <ArgsFormales> <Bloque> en sus cuatro apariciones, incluida la de <Constructor>, y <Metodo> ::= idMetVar <CuerpoDeMetodo> en las ramas static y void.

Resultado

<Miembro> ::= <Tipo> idMetVar <AtributoOMetodo> | static <TipoMetodo>
<Metodo> | void <Metodo> | <Constructor>

<AtributoOMetodo> ::= ; | <CuerpoDeMetodo>

<Metodo> ::= idMetVar <CuerpoDeMetodo>

<CuerpoDeMetodo> ::= <ArgsFormales> <Bloque>

<Constructor> ::= public idClase <CuerpoDeMetodo>

Se pierden <Atributo> y <ModificadorOpcional>. <Metodo> conserva el nombre pero deriva la cola de las ramas static y void; el método con tipo y sin static llega por <AtributoOMetodo>, porque su idMetVar se consumió al factorizar contra el atributo.

F7 — <Primario>, prefijo idMetVar

Original

<Primario> ::= this | stringLiteral | <AccesoVar> | <CrearArreglo> |
<LlamadaConstructor> | <LlamadaMetodo> | <LlamadaMetodoEstatico> |
<ExpresionParentizada>

<AccesoVar> ::= idMetVar

<LlamadaMetodo> ::= idMetVar <ArgsActuales>

Conflicto. <AccesoVar> y <LlamadaMetodo> en {idMetVar}. El conflicto en {new} se trata en F8.

Paso 1 (AG). <AccesoVarOLlamadaMetodo> ::= <AccesoVar> | <LlamadaMetodo> reemplaza a las dos en <Primario>.

Paso 2. **FP**: **SU** de las dos, que da <AccesoVarOLlamadaMetodo> ::= idMetVar | idMetVar <ArgsActuales>, y **FD** con $\alpha =$ idMetVar, $\beta_1 = \epsilon$, $\beta_2 =$ <ArgsActuales> y $R =$ <AccesoOLlamada>.

Resultado

<AccesoVarOLlamadaMetodo> ::= idMetVar <AccesoOLlamada>

<AccesoOLlamada> ::= <ArgsActuales> | ϵ

{(} contra {ϵ}. ✓ No se comparte con <EncadenadoAtributoOMetodo> aunque tengan la misma forma: su ϵ es una variable propia, y el de aquél un atributo de otro objeto.

F8 — <Primario>, prefijo new

Original

<CrearArreglo> ::= new <TipoBase> <DimensionesConTamanio>

<LlamadaConstructor> ::= new <TipoReferencia> <ArgsActuales>

<TipoBase> ::= <TipoPrimitivo> | <TipoReferencia> | idGen

Conflicto. {new} contra {new}.

Paso 1 (AG). <Creacion> ::= <CrearArreglo> | <LlamadaConstructor> reemplaza a las dos en <Primario>.

Paso 2. **FP**: **SU** de las dos y **FD** con $\alpha =$ new y $R =$ <ArregloOInstancia>:

<Creacion> ::= new <ArregloOInstancia>

<ArregloOInstancia> ::= <TipoBase> <DimensionesConTamanio> |
<TipoReferencia> <ArgsActuales>

Conflicto nuevo: $\mathcal{T}$ contra {idClase}.

Paso 3. **FP**: **SU** de <TipoBase>,

<ArregloOInstancia> ::= <TipoPrimitivo> <DimensionesConTamanio> |
<TipoReferencia> <DimensionesConTamanio> | idGen
<DimensionesConTamanio> | <TipoReferencia> <ArgsActuales>

y **FD** con $\alpha =$ <TipoReferencia>, $\beta_1 =$ <DimensionesConTamanio>, $\beta_2 =$ <ArgsActuales> y $R =$ <ArregloOInstanciaDeClase>. <TipoReferencia> no se sube: no hace falta.

Resultado

<Primario> ::= this | stringLiteral | <ExpresionParentizada> |
<AccesoVarOLlamadaMetodo> | <LlamadaMetodoEstatico> | <Creacion>

<Creacion> ::= new <ArregloOInstancia>

<ArregloOInstancia> ::= <TipoPrimitivo> <DimensionesConTamanio> | idGen
<DimensionesConTamanio> | <TipoReferencia> <ArregloOInstanciaDeClase>

<ArregloOInstanciaDeClase> ::= <DimensionesConTamanio> | <ArgsActuales>

<Primario>: {this}, {stringLiteral}, {(}, {idMetVar}, {idClase} y {new}.
<ArregloOInstancia>: {boolean, char, int}, {idGen} y {idClase}.
<ArregloOInstanciaDeClase>: {[} contra {(}. ✓ Se pierden <AccesoVar>, <LlamadaMetodo>, <CrearArreglo> y <LlamadaConstructor>.

3.5 El núcleo

Con R1–R3 y F1–F8 la gramática es LL(1): de 60 a 67 no terminales y de 126 a 135 producciones. Genera el mismo lenguaje que la de la cátedra. Los diez no terminales que se pierden lo hacen por **SU**.

3.6 Logros

Cada logro parte de la gramática que dejó el anterior: primero se escribe su extensión y después se la lleva a LL(1) con las mismas técnicas.

L1 — Floats (Etapa 1)

Extensión

<TipoPrimitivo> ::= **boolean** | **char** | **int** | **float**

<Primitivo> ::= **true** | **false** | **intLiteral** | **charLiteral** | **null** | **floatLiteral**

Sin conflictos.

L2 — Operadores Posifijos

Extensión

<Operando> ::= <Primitivo> | <Referencia> <PosfijoOpcional>

<PosfijoOpcional> ::= ++ | -- | ϵ

<Operando>: los literales contra $\mathcal{P}$. <PosfijoOpcional>: {++}, {--} y {ϵ}. Sin conflictos.

L3 — Operador Ternario

Extensión Sobre las reglas de la base, el lado derecho del = y el de <VarLocal> pasan a ser una <ExpresionCondicional>, con los operandos del JLS §15.25:

<Expresion> ::= <ExpresionCompuesta> <OperadorAsignacion>
    <ExpresionCondicional> | <ExpresionCondicional>

<ExpresionCondicional> ::= <ExpresionCompuesta> **?** <Expresion> **:**
    <ExpresionCondicional> | <ExpresionCompuesta>

<VarLocal> ::= **var idMetVar =** <ExpresionCondicional>

Conflictos. En <ExpresionCondicional> y en <Expresion> las dos alternativas tienen Primeros $\mathcal{E}$.

Paso 1. **FD** en <ExpresionCondicional> con α = <ExpresionCompuesta> y R = <RestoCondicional>.

Paso 2. **FP** en <Expresion>: **SU** de <ExpresionCondicional> en la segunda, que queda <ExpresionCompuesta> <RestoCondicional>, y **FD** con α = <ExpresionCompuesta>, β_1 = <OperadorAsignacion> <ExpresionCondicional>, β_2 = <RestoCondicional> y R = <RestoExpresion>, que reemplaza al de F1.

Resultado

<Expresion> ::= <ExpresionCompuesta> <RestoExpresion>

<RestoExpresion> ::= <OperadorAsignacion> <ExpresionCondicional> |
    <RestoCondicional>

<ExpresionCondicional> ::= <ExpresionCompuesta> <RestoCondicional>

<RestoCondicional> ::= **?** <Expresion> **:** <ExpresionCondicional> | ϵ

<RestoExpresion>: {=} contra {?, ϵ}. <RestoCondicional>: {?} contra {ϵ}. ✓ El lado izquierdo del = sigue siendo una <ExpresionCompuesta> y el derecho no admite otro =. El ternario es asociativo a derecha.

L4 — Variables Locales Clásicas

Extensión

<Sentencia> ::= ... | <VarLocalClasica> **;**

<VarLocalClasica> ::= <Tipo> <ListaDeclaradores>

<ListaDeclaradores> ::= idMetVar <InicializacionDeDeclarador> |
<ListaDeclaradores> , idMetVar <InicializacionDeDeclarador>

<InicializacionDeDeclarador> ::= = <ExpresionCondicional> | ϵ

Conflictos. <ListaDeclaradores> tiene recursión a izquierda. En <Sentencia>, <SentenciaExpresion> ; ($\mathcal{E}$) y <VarLocalClasica> ; ($\mathcal{T}$) chocan en {idClase}: `C.m();` contra `C x;`.

Paso 1. **RI** en <ListaDeclaradores> con β_1 = idMetVar <InicializacionDeDeclarador>, α_1 = , idMetVar <InicializacionDeDeclarador> y R = <RestoListaDeclaradores>.

Paso 2. **FP** en <Sentencia>: **SU** hasta exponer idClase. La declaración se sube por <VarLocalClasica>, <Tipo>, <TipoBase> y <TipoReferencia>; la expresión, por <SentenciaExpresion>, <Expresion>, <ExpresionCompuesta>, <ExpresionBasica>, <Operando>, <Referencia>, <Primario> y <LlamadaMetodoEstatico>. Con X cada uno de this, stringLiteral, <ExpresionParentizada>, <AccesoVarOLlamadaMetodo> y <Creacion>, esas dos alternativas pasan a ser:

<Sentencia> ::= <TipoPrimitivo> <DimensionesOpcionales>
<ListaDeclaradores> ;

<Sentencia> ::= idGen <DimensionesOpcionales> <ListaDeclaradores> ;

<Sentencia> ::= idClase <TipoGenericoOpcional> <DimensionesOpcionales>
<ListaDeclaradores> ;

<Sentencia> ::= <OperadorUnario> <Operando> <RestoExpresionCompuesta>
<RestoExpresion> ;

<Sentencia> ::= <Primitivo> <RestoExpresionCompuesta> <RestoExpresion> ;

<Sentencia> ::= X <RestoReferencia> <PosfijoOpcional>
<RestoExpresionCompuesta> <RestoExpresion> ;

<Sentencia> ::= idClase . idMetVar <ArgsActuales> <RestoReferencia>
<PosfijoOpcional> <RestoExpresionCompuesta> <RestoExpresion> ;

Paso 3. **FD** con α = idClase y R = <SentenciaDesdeIdClase>.

Paso 4 (AG). Para no duplicar las colas:

<ColaTrasOperando> ::= <RestoExpresionCompuesta> <RestoExpresion>

<ColaDeExpresionDesdePrimario> ::= <RestoReferencia> <PosfijoOpcional>
<ColaTrasOperando>

<PrimarioSinIdClase> ::= this | stringLiteral | <ExpresionParentizada> |
<AccesoVarOLlamadaMetodo> | <Creacion>

<Primario> ::= <LlamadaMetodoEstatico> | <PrimarioSinIdClase>

<ExpresionSinIdClase> ::= <OperadorUnario> <Operando> <ColaTrasOperando> |
<Primitivo> <ColaTrasOperando> | <PrimarioSinIdClase>
<ColaDeExpresionDesdePrimario>

Resultado

<Sentencia> ::= ; | <VarLocal> ; | <Return> ; | <If> | <While> | <Bloque> |
<ExpresionSinIdClase> ;

<Sentencia> ::= <TipoPrimitivo> <DimensionesOpcionales>
<ListaDeclaradores> ; | idGen <DimensionesOpcionales>
<ListaDeclaradores> ; | idClase <SentenciaDesdeIdClase>

<SentenciaDesdeIdClase> ::= <TipoGenericoOpcional> <DimensionesOpcionales> <ListaDeclaradores> ; | . idMetVar <ArgsActuales> <ColaDeExpresionDesdePrimario> ;

<ListaDeclaradores> ::= idMetVar <InicializacionDeDeclarador> <RestoListaDeclaradores>

<RestoListaDeclaradores> ::= , idMetVar <InicializacionDeDeclarador> <RestoListaDeclaradores> | ϵ

más los cinco no terminales del paso 4. <Sentencia>: {;}, {var}, {return}, {if}, {while}, {{}, $\mathcal{E}$ sin idClase, {boolean, char, float, int}, {idGen} y {idClase}. <SentenciaDesdeIdClase>: {<, [, idMetVar} contra {.}. <Primario>: {idClase} contra el resto de $\mathcal{P}$. ✓ Se disuelven <SentenciaExpresion> y <VarLocalClasica>. Mirar dos tokens después del idClase evitaría los no terminales nuevos, pero se aparta del Esquema Simple, que usa uno.

L5 — Atributos Inicializados

Extensión La cola del atributo admite la inicialización:

<AtributoOMetodo> ::= <InicializacionDeAtributo> ; | <CuerpoDeMetodo>

<InicializacionDeAtributo> ::= = <ExpresionCondicional> | ϵ

{=, ;} contra {(}. Sin conflictos. No se comparte con <InicializacionDeDeclarador>: para el semántico son construcciones distintas.

L6 — Visibilidad Mejorada

Extensión Visibilidad opcional en atributos, métodos y constructores; static antes o después de ella, sin repetir ninguno; public opcional en los métodos de interfaz:

<Miembro> ::= <VisibilidadOpcional> <Tipo> idMetVar <AtributoOMetodo> | <Visibilidad> static <TipoMetodo> <Metodo>

<Miembro> ::= static <VisibilidadOpcional> <TipoMetodo> <Metodo> | <VisibilidadOpcional> void <Metodo> | <Constructor>

<Constructor> ::= <VisibilidadOpcional> idClase <CuerpoDeMetodo>

<Visibilidad> ::= public | private

<VisibilidadOpcional> ::= <Visibilidad> | ϵ

<MetodoInterfaz> ::= <PublicOpcional> <TipoMetodo> idMetVar <ArgsFormales> ;

<PublicOpcional> ::= public | ϵ

Conflictos. En <Miembro>, {public, private} en la 1ª, 2ª, 4ª y 5ª, e {idClase} en la 1ª y la 5ª.

Paso 1. **SU** de <VisibilidadOpcional> en la 1ª y la 4ª, y de <Constructor>:

<Miembro> ::= <Visibilidad> <Tipo> idMetVar <AtributoOMetodo> | <Tipo> idMetVar <AtributoOMetodo>

<Miembro> ::= <Visibilidad> static <TipoMetodo> <Metodo> | static <VisibilidadOpcional> <TipoMetodo> <Metodo>

<Miembro> ::= <Visibilidad> **void** <Metodo> | **void** <Metodo>

<Miembro> ::= <Visibilidad> **idClase** <CuerpoDeMetodo> | **idClase** <CuerpoDeMetodo>

Paso 2 (AG). Las tres construcciones que aparecen con y sin visibilidad:

<MiembroSinModificadores> ::= **void** <Metodo> | <Tipo> **idMetVar** <AtributoOMetodo> | **idClase** <CuerpoDeMetodo>

<Miembro> ::= <Visibilidad> <MiembroSinModificadores> | <Visibilidad> **static** <TipoMetodo> <Metodo> | **static** <VisibilidadOpcional> <TipoMetodo> <Metodo> | <MiembroSinModificadores>

Paso 3. **FD** en <Miembro> con α = <Visibilidad> y R = <MiembroTrasVisibilidad>.

Paso 4. En <MiembroSinModificadores> chocan $\mathcal{T}$ y {**idClase**}: **FP** con **SU** de <Tipo>, <TipoBase> y <TipoReferencia>, y **FD** con α = **idClase** y R = <MiembroDesdeIdClase>.

Resultado

<Miembro> ::= <Visibilidad> <MiembroTrasVisibilidad> | **static** <VisibilidadOpcional> <TipoMetodo> <Metodo> | <MiembroSinModificadores>

<MiembroTrasVisibilidad> ::= **static** <TipoMetodo> <Metodo> | <MiembroSinModificadores>

<MiembroSinModificadores> ::= **void** <Metodo> | <TipoPrimitivo> <DimensionesOpcionales> **idMetVar** <AtributoOMetodo>

<MiembroSinModificadores> ::= **idGen** <DimensionesOpcionales> **idMetVar** <AtributoOMetodo> | **idClase** <MiembroDesdeIdClase>

<MiembroDesdeIdClase> ::= <CuerpoDeMetodo> | <TipoGenericoOpcional> <DimensionesOpcionales> **idMetVar** <AtributoOMetodo>

<Miembro>: {**public**, **private**}, {**static**} y {**boolean**, **char**, **float**, **int**, **idClase**, **idGen**, **void**}. <MiembroTrasVisibilidad>: {**static**} contra ese último. <MiembroSinModificadores>: {**void**}, {**boolean**, **char**, **float**, **int**}, {**idGen**} y {**idClase**}. <MiembroDesdeIdClase>: {**(**} contra {**<**, **[**, **idMetVar**}. ✓ Se disuelve <Constructor>: sin un **public** obligatorio, compite con el tipo por **idClase**.

L7 — Fors!

Extensión

<Sentencia> ::= ... | <For>

<For> ::= **for (** <EncabezadoFor> **)** <Sentencia>

<EncabezadoFor> ::= <VarLocal> <ClausulasFinalesFor> | <Expresion> <ClausulasFinalesFor> | **var idMetVar :** <Expresion>

<ClausulasFinalesFor> ::= **;** <Expresion> **;** <Expresion>

Conflicto. En <EncabezadoFor>, {**var**} contra {**var**}.

Paso. **FP**: **SU** de <VarLocal>, que da **var idMetVar =** <ExpresionCondicional> <ClausulasFinalesFor>, y **FD** con α = **var idMetVar** y R = <EncabezadoForTrasVar>.

Resultado

<EncabezadoFor> ::= var idMetVar <EncabezadoForTrasVar> | <Expresion> <ClausulasFinalesFor>

<EncabezadoForTrasVar> ::= : <Expresion> | = <ExpresionCondicional> <ClausulasFinalesFor>

<EncabezadoFor>: {var} contra $\mathcal{E}$. <EncabezadoForTrasVar>: {:} contra {=}. ✓

L8 — Inicialización de Arreglos

Extensión Donde F8 dejó <DimensionesConTamanio> va una <CreacionDeArreglo>, que admite además la forma de Java con inicializador (JLS §15.10.1 y §10.6):

<ArregloOInstancia> ::= <TipoPrimitivo> <CreacionDeArreglo> | idGen <CreacionDeArreglo> | <TipoReferencia> <ArregloOInstanciaDeClase>

<ArregloOInstanciaDeClase> ::= <CreacionDeArreglo> | <ArgsActuales>

<CreacionDeArreglo> ::= <DimensionesConTamanio> | [ ] <DimensionesOpcionales> <InicializadorDeArreglo>

<InicializadorDeArreglo> ::= { <ListaElementosOpcional> }

<ListaElementosOpcional> ::= <ListaElementos> | , | ϵ

<ListaElementos> ::= <ElementoInicializador> | <ElementoInicializador> , | <ElementoInicializador> , <ListaElementos>

<ElementoInicializador> ::= <Expresion> | <InicializadorDeArreglo>

Conflictos. En <CreacionDeArreglo>, {[} contra {[}. En <ListaElementos>, las tres alternativas tienen Primeros $\mathcal{E} \cup \{\{\}$.

Paso 1. **FP** en <CreacionDeArreglo>: **SU** de <DimensionesConTamanio>, que da [<Expresion>] <DimensionesConTamanioOpcional>, y **FD** con α = [y R = <RestoCreacionDeArreglo>.

Paso 2. **FD** en <ListaElementos> con α = <ElementoInicializador> y R = <RestoListaElementos>, que queda <RestoListaElementos> ::= ϵ | , | , <ListaElementos>: conflicto {,} contra {,}.

Paso 3. **FD** en <RestoListaElementos> con α = ,, $\beta_1 = \epsilon$, β_2 = <ListaElementos> y R = <ElementosTrasComa>.

Resultado

<CreacionDeArreglo> ::= [<RestoCreacionDeArreglo>

<RestoCreacionDeArreglo> ::= <Expresion>] <DimensionesConTamanioOpcional> |] <DimensionesOpcionales> <InicializadorDeArreglo>

<ListaElementos> ::= <ElementoInicializador> <RestoListaElementos>

<RestoListaElementos> ::= , <ElementosTrasComa> | ϵ

<ElementosTrasComa> ::= <ListaElementos> | ϵ

<RestoCreacionDeArreglo>: $\mathcal{E}$ contra {]}. <ListaElementosOpcional>: $\mathcal{E} \cup \{\{\}$, {,} y {ϵ}. <RestoListaElementos>: {,} contra {ϵ}. <ElementosTrasComa>: $\mathcal{E} \cup \{\{\}$ contra {ϵ}. <ElementoInicializador>: $\mathcal{E}$ contra {{}. ✓ Sin inicializador sigue la creación de la base, con toda dimensión con su tamaño. <InicializadorDeArreglo> sólo es alcanzable desde un new.

L9 — Genericidad Avanzada

Extensión, anidamiento. `<InstanciadoOParametrico> ::=` **idGen** | **idClase** `<TipoGenericoOpcional>`. {**idGen**} contra {**idClase**}: sin conflictos.

Extensión, diamante. Sólo al instanciar: `<ArregloOInstancia> ::= ...` | **idClase** < > `<ArgsActuales>`.

Conflicto. `<TipoReferencia>` `<ArregloOInstanciaDeClase>` contra **idClase** < > `<ArgsActuales>`, en {**idClase**}.

Paso 1. **FP**: **SU** de `<TipoReferencia>`, que da **idClase** `<TipoGenericoOpcional>` `<ArregloOInstanciaDeClase>`, y **FD** con α = **idClase** y R = `<CreacionDesdeIdClase>`, que queda `<CreacionDesdeIdClase> ::=` `<TipoGenericoOpcional>` `<ArregloOInstanciaDeClase>` | < > `<ArgsActuales>`: conflicto en {<}.

Paso 2. **FP**: **SU** de `<TipoGenericoOpcional>`, que da < `<InstanciadoOParametrico>` > `<ArregloOInstanciaDeClase>` | `<ArregloOInstanciaDeClase>`, y **FD** con α = < y R = `<GenericoDeCreacion>`.

Resultado

```
<ArregloOInstancia> ::= <TipoPrimitivo> <CreacionDeArreglo> | idGen
    <CreacionDeArreglo> | idClase <CreacionDesdeIdClase>
<CreacionDesdeIdClase> ::= < <GenericoDeCreacion> |
    <ArregloOInstanciaDeClase>
<GenericoDeCreacion> ::= > <ArgsActuales> | <InstanciadoOParametrico> >
    <ArregloOInstanciaDeClase>
```

`<CreacionDesdeIdClase>`: {<} contra {**[**, **(**}. `<GenericoDeCreacion>`: {>} contra {**idClase**, **idGen**}. ✓ `List<> x;` y `new C<>[3]` no derivan: el diamante sólo lleva a `<ArgsActuales>`.

3.7 Ambigüedades que quedan

La (1) se eliminó en F2. Las otras dos se resuelven por regla de implementación, y de la misma forma: el analizador toma la alternativa no vacía apenas ve su token.

No terminal	Caso	Resolución
`<ElseOpcional>`	`if (a) if (b) s1; else s2;`	el **else** liga al **if** más cercano
`<DimensionesConTamanioOpcional>`	`new int[3][4]`	el **[** es otra dimensión

Las dos coinciden con Java. Detectarlas exige mirar qué sigue a un no terminal anulable; ese cálculo es una auditoría hecha fuera del compilador, que decide sólo con Primeros.

4 Gramática final

```
<Inicial> ::= <ListaClases> eof
<ListaClases> ::= <Clase> <ListaClases> | <Interfaz> <ListaClases> | ϵ
```

<Clase> ::= **class idClase** <GenericidadOpcional> <HerenciaOpcional> **{** <ListaMiembros> **}**

<Interfaz> ::= **interface idClase** <GenericidadOpcional> <ExtensionOpcional> **{** <ListaMetodosInterfaz> **}**

<GenericidadOpcional> ::= **<** **idGen** **>** | ϵ

<HerenciaOpcional> ::= **extends** <TipoReferencia> | **implements** <TipoReferencia> | ϵ

<ExtensionOpcional> ::= **extends** <TipoReferencia> | ϵ

<ListaMiembros> ::= <Miembro> <ListaMiembros> | ϵ

<ListaMetodosInterfaz> ::= <MetodoInterfaz> <ListaMetodosInterfaz> | ϵ

<Miembro> ::= <Visibilidad> <MiembroTrasVisibilidad>

<Miembro> ::= **static** <VisibilidadOpcional> <TipoMetodo> <Metodo>

<Miembro> ::= <MiembroSinModificadores>

<MiembroTrasVisibilidad> ::= **static** <TipoMetodo> <Metodo>

<MiembroTrasVisibilidad> ::= <MiembroSinModificadores>

<Visibilidad> ::= **public** | **private**

<VisibilidadOpcional> ::= <Visibilidad> | ϵ

<MiembroSinModificadores> ::= **void** <Metodo>

<MiembroSinModificadores> ::= <TipoPrimitivo> <DimensionesOpcionales> **idMetVar** <AtributoOMetodo>

<MiembroSinModificadores> ::= **idGen** <DimensionesOpcionales> **idMetVar** <AtributoOMetodo>

<MiembroSinModificadores> ::= **idClase** <MiembroDesdeIdClase>

<MiembroDesdeIdClase> ::= <CuerpoDeMetodo>

<MiembroDesdeIdClase> ::= <TipoGenericoOpcional> <DimensionesOpcionales> **idMetVar** <AtributoOMetodo>

<AtributoOMetodo> ::= <InicializacionDeAtributo> **;**

<AtributoOMetodo> ::= <CuerpoDeMetodo>

<InicializacionDeAtributo> ::= **=** <ExpresionCondicional>

<InicializacionDeAtributo> ::= ϵ

<Metodo> ::= **idMetVar** <CuerpoDeMetodo>

<CuerpoDeMetodo> ::= <ArgsFormales> <Bloque>

<MetodoInterfaz> ::= <PublicOpcional> <TipoMetodo> **idMetVar** <ArgsFormales> **;**

<PublicOpcional> ::= **public**

<PublicOpcional> ::= ϵ

<TipoMetodo> ::= <Tipo> | **void**

<Tipo> ::= <TipoBase> <DimensionesOpcionales>

<TipoBase> ::= <TipoPrimitivo> | <TipoReferencia> | **idGen**

<DimensionesOpcionales> ::= [ ] <DimensionesOpcionales> | ϵ
<TipoReferencia> ::= idClase <TipoGenericoOpcional>

<TipoPrimitivo> ::= boolean | char | int
<TipoPrimitivo> ::= float

<TipoGenericoOpcional> ::= < <InstanciadoOParametrico> > | ϵ

<InstanciadoOParametrico> ::= idGen
<InstanciadoOParametrico> ::= idClase <TipoGenericoOpcional>
<ArgsFormales> ::= (<ListaArgsFormalesOpcional>)
<ListaArgsFormalesOpcional> ::= <ListaArgsFormales> | ϵ
<ListaArgsFormales> ::= <ArgFormal> <RestoListaArgsFormales>
<RestoListaArgsFormales> ::= , <ArgFormal> <RestoListaArgsFormales> | ϵ
<ArgFormal> ::= <Tipo> idMetVar

<Bloque> ::= { <ListaSentencias> }
<ListaSentencias> ::= <Sentencia> <ListaSentencias> | ϵ
<Sentencia> ::= ;
<Sentencia> ::= <VarLocal> ;
<Sentencia> ::= <Return> ;
<Sentencia> ::= <If>
<Sentencia> ::= <While>
<Sentencia> ::= <Bloque>
<Sentencia> ::= <For>
<Sentencia> ::= <ExpresionSinIdClase> ;
<Sentencia> ::= <TipoPrimitivo> <DimensionesOpcionales>
    <ListaDeclaradores> ;
<Sentencia> ::= idGen <DimensionesOpcionales> <ListaDeclaradores> ;
<Sentencia> ::= idClase <SentenciaDesdeIdClase>
<SentenciaDesdeIdClase> ::= <TipoGenericoOpcional> <DimensionesOpcionales>
    <ListaDeclaradores> ;
<SentenciaDesdeIdClase> ::= . idMetVar <ArgsActuales>
    <ColaDeExpresionDesdePrimario> ;
<VarLocal> ::= var idMetVar = <ExpresionCondicional>

<ListaDeclaradores> ::= idMetVar <InicializacionDeDeclarador>
    <RestoListaDeclaradores>
<RestoListaDeclaradores> ::= , idMetVar <InicializacionDeDeclarador>
    <RestoListaDeclaradores>
<RestoListaDeclaradores> ::= ϵ
<InicializacionDeDeclarador> ::= = <ExpresionCondicional> | ϵ
<Return> ::= return <ExpresionOpcional>
<ExpresionOpcional> ::= <Expresion> | ϵ
<If> ::= if (<Expresion>) <Sentencia> <ElseOpcional>
<ElseOpcional> ::= else <Sentencia> | ϵ

<While> ::= while (<Expresion>) <Sentencia>

<For> ::= for (<EncabezadoFor>) <Sentencia>
<EncabezadoFor> ::= var idMetVar <EncabezadoForTrasVar>
<EncabezadoFor> ::= <Expresion> <ClausulasFinalesFor>

<EncabezadoForTrasVar> ::= : <Expresion>
<EncabezadoForTrasVar> ::= = <ExpresionCondicional> <ClausulasFinalesFor>

<ClausulasFinalesFor> ::= ; <Expresion> ; <Expresion>

<Expresion> ::= <ExpresionCompuesta> <RestoExpresion>

<RestoExpresion> ::= <OperadorAsignacion> <ExpresionCondicional>
<RestoExpresion> ::= <RestoCondicional>
<OperadorAsignacion> ::= =

<ExpresionCondicional> ::= <ExpresionCompuesta> <RestoCondicional>

<RestoCondicional> ::= ? <Expresion> : <ExpresionCondicional>
<RestoCondicional> ::= ϵ
<ExpresionCompuesta> ::= <ExpresionBasica> <RestoExpresionCompuesta>
<RestoExpresionCompuesta> ::= <OperadorBinario> <ExpresionBasica>
    <RestoExpresionCompuesta> | ϵ
<OperadorBinario> ::= || | && | == | != | < | > | <= | >= | + | - |
    * | / | %
<ExpresionBasica> ::= <OperadorUnario> <Operando>
<ExpresionBasica> ::= <Operando>
<OperadorUnario> ::= + | - | !
<Operando> ::= <Primitivo>
<Operando> ::= <Referencia> <PosfijoOpcional>

<PosfijoOpcional> ::= ++
<PosfijoOpcional> ::= --
<PosfijoOpcional> ::= ϵ
<Primitivo> ::= true | false | intLiteral | charLiteral | null
<Primitivo> ::= floatLiteral

<ExpresionSinIdClase> ::= <OperadorUnario> <Operando> <ColaTrasOperando>
<ExpresionSinIdClase> ::= <Primitivo> <ColaTrasOperando>
<ExpresionSinIdClase> ::= <PrimarioSinIdClase>
    <ColaDeExpresionDesdePrimario>
<ColaTrasOperando> ::= <RestoExpresionCompuesta> <RestoExpresion>
<ColaDeExpresionDesdePrimario> ::= <RestoReferencia> <PosfijoOpcional>
    <ColaTrasOperando>
<Referencia> ::= <Primario> <RestoReferencia>
<RestoReferencia> ::= <Encadenado> <RestoReferencia>
<RestoReferencia> ::= <AccesoArreglo> <RestoReferencia>

<RestoReferencia> ::= ϵ

<Encadenado> ::= **. idMetVar** <EncadenadoAtributoOMetodo>

<EncadenadoAtributoOMetodo> ::= <ArgsActuales> | ϵ

<AccesoArreglo> ::= **[** <Expresion> **]**

<Primario> ::= <LlamadaMetodoEstatico>

<Primario> ::= <PrimarioSinIdClase>

<LlamadaMetodoEstatico> ::= **idClase . idMetVar** <ArgsActuales>

<PrimarioSinIdClase> ::= **this**

<PrimarioSinIdClase> ::= **stringLiteral**

<PrimarioSinIdClase> ::= <ExpresionParentizada>

<PrimarioSinIdClase> ::= <AccesoVarOLlamadaMetodo>

<PrimarioSinIdClase> ::= <Creacion>

<ExpresionParentizada> ::= **(** <Expresion> **)**

<AccesoVarOLlamadaMetodo> ::= **idMetVar** <AccesoOLlamada>

<AccesoOLlamada> ::= <ArgsActuales> | ϵ

<Creacion> ::= **new** <ArregloOInstancia>

<ArregloOInstancia> ::= <TipoPrimitivo> <CreacionDeArreglo>

<ArregloOInstancia> ::= **idGen** <CreacionDeArreglo>

<ArregloOInstancia> ::= **idClase** <CreacionDesdeIdClase>

<CreacionDesdeIdClase> ::= < <GenericoDeCreacion>

<CreacionDesdeIdClase> ::= <ArregloOInstanciaDeClase>

<GenericoDeCreacion> ::= > <ArgsActuales>

<GenericoDeCreacion> ::= <InstanciadoOParametrico> >
<ArregloOInstanciaDeClase>

<ArregloOInstanciaDeClase> ::= <CreacionDeArreglo>

<ArregloOInstanciaDeClase> ::= <ArgsActuales>

<CreacionDeArreglo> ::= **[** <RestoCreacionDeArreglo>

<RestoCreacionDeArreglo> ::= <Expresion> **]** <DimensionesConTamanioOpcional>

<RestoCreacionDeArreglo> ::= **]** <DimensionesOpcionales>
<InicializadorDeArreglo>

<DimensionesConTamanio> ::= **[** <Expresion> **]** <DimensionesConTamanioOpcional>

<DimensionesConTamanioOpcional> ::= <DimensionesConTamanio> | ϵ

<InicializadorDeArreglo> ::= **{** <ListaElementosOpcional> **}**

<ListaElementosOpcional> ::= <ListaElementos> | **,** | ϵ

<ListaElementos> ::= <ElementoInicializador> <RestoListaElementos>

<RestoListaElementos> ::= **,** <ElementosTrasComa> | ϵ

<ElementosTrasComa> ::= <ListaElementos> | ϵ

<ElementoInicializador> ::= <Expresion> | <InicializadorDeArreglo>

<ArgsActuales> ::= **(** <ListaExpsOpcional> **)**

<ListaExpsOpcional> ::= <ListaExps> | ϵ

```
<ListaExps> ::= <Expresion> <RestoListaExps>
<RestoListaExps> ::= , <ListaExps> | ϵ
```

5 El analizador sintáctico

5.1 Diseño

Descendente recursivo predictivo con el Esquema Simple de Clase 5: un método por no terminal, `match` para los terminales, y la producción se elige mirando si el token actual está en los Primeros de su cuerpo. Si ninguna coincide, el `else` final lanza el error; si alguna parte derecha deriva a ϵ, es un `else` vacío con esa producción citada. La recursión a derecha se traduce con recursión.

Clase	Responsabilidad
`AnalizadorSintactico`	Carga el primer token, llama a `<Inicial>` y devuelve los errores.
`EntradaDeTokens`	El token actual y `match`: lo único que consume entrada.
`Sintaxis`	Conecta las nueve secciones, que son mutuamente recursivas.
`partes/AnalizadorDe{Clases, Miembros, Tipos, Sentencias, Fors, Expresiones, Referencias, Creacion, Argumentos}`	Un método por no terminal de cada sección de la gramática.
`primeros/PrimerosDeX`	Los Primeros de cada producción, generados desde la gramática.
`recuperacion/*`	Modo Pánico.
`errores/ExcepcionSintactica`	El error sintáctico.

5.2 Errores que detecta

El Esquema Simple falla en dos lugares: un `match` que no coincide (se esperaba un terminal y llegó otro) y un no terminal no anulable cuyo token actual no está en los Primeros de ninguna de sus producciones (se esperaba alguno de esos terminales). El mensaje nombra lo esperado y lo encontrado:

```
Error Sintactico en linea 3: se esperaba idClase se encontro "clasesita"

[Error:clasesita|3]
```

El lexema del fin de archivo es `$`. Con Modo Pánico, ante un error se descarta entrada hasta un `;`, una `}`, una `{` que no abre un elemento nuevo, o un token que empiece el siguiente elemento de la lista (clases, miembros, métodos de interfaz o sentencias), y se reporta cada error en orden. El primero es siempre el mismo que sin recuperación. Los errores léxicos los reporta el analizador de la Etapa 1.

6 Decisiones

6.1 Sobre puntos consultados

Tema	Decisión
Huecos del **for**	Una expresión en cada uno, obligatoria: `for(;;)` y `for(var i = 0; i < n; )` no son válidos.
Variable de control del **for**	Sólo con **var**: `for (int i = 0; ...)` y `for (Persona p : lista)` no son válidos.
Asignaciones anidadas	El lado derecho de un **=**, en la asignación y en los inicializadores, es una `<ExpresionCondicional>`: `x = y = 2` e `int x = y = 2;` no son válidos; `x = (y = 2)` sí.
Inicialización de arreglos	Como en Java: con inicializador ninguna dimensión lleva tamaño, se anidan y admiten coma final. Sin inicializador rige la base, con toda dimensión con tamaño: `new int[2][]` no es válido.
Atributos	**static** sólo modifica métodos, y un atributo por declaración: `static int x;` e `int a, b;` no son válidos.
Modificadores	**static** y la visibilidad en cualquier orden, sin repetir; **protected** no existe.
Logros	No son configurables.
Recursión a derecha	Se traduce con recursión, no con `while`.
`else` final	Ningún método de no terminal lo omite.
Lexema del EOF	**$**.

6.2 Otras

Tema	Decisión
Métodos de interfaz	Admiten **public** y no **private**: Visibilidad Mejorada nombra a los métodos, y en Java un método de interfaz privado exige cuerpo, que `<MetodoInterfaz>` no tiene.
Operadores posfijos	Sobre una `<Referencia>` y uno solo: `5++` y `x++ ++` no son válidos. Que la referencia sea una variable lo verifica el semántico.
float	**float** y **floatLiteral** entran a `<TipoPrimitivo>` y `<Primitivo>`, para que el logro Floats I tenga uso en la sintaxis.
Ternario	Operandos del JLS §15.25 y asociativo a derecha: `a ? b = 1 : c` es válido y `a ? b : c = d` no.